

Gulliver Language

Uma Abordagem Probabilística

Por: Lucas Barbosa Brancalhão

Introdução

A linguagem gulliver é uma linguagem feita para processos funcionais. Ela tem como objetivo colaborar para a resolução de problemas e modelos matemáticos, especialmente probabilísticos, assim como a programação quântica, sendo essa última uma generalização especial de problemas matemáticos probabilísticos.

Abaixo será descrita a estrutura da linguagem, assim como os tipos de dados, palavras reservadas, operadores lógicos, estruturas de repetição, condicionais, entre outros.

Palavras Reservadas

Comandos	Definição
int	Tipo de dados: inteiro
string	Tipo de dados: texto
bool	Tipo de dados: booleano
float	Tipo de dados: ponto flutuante
def	Declaração de função
function	Tipo de dados: função
if	Condicional “Se”
else	Condicional “Se não”
print	Imprima
main	Função Corpo
read	leia
complex	Tipo de dados: números complexos
double	Tipo de dados: Double

Exemplo de código

Função HelloWorld: Todo programa deve começar com uma função main, que irá conter o código executado. Abaixo, a função main contém a função HelloWorld.

```
def main () -> void {
    def HelloWorld() -> void{
        print("Olá Mundo")
    }
}
```

Função Fatorial: Abaixo um exemplo de um programa Fatorial. Programas que necessitam de repetição devem usar o princípio de recursão.

```
def main () -> void {
    def fatorial(parametro: int) -> int {
        if(parametro == 0) {
            return 1;
        } else if(parametro > 0) {
            return fatorial(parametro - 1);
        } else {
            return -1;
        }
    }
}
```

Declaração de variável: Abaixo uma declaração de variável.

```
let variavel: string = "Exemplo de Texto";
```

Operadores

+	Soma
-	Subtração
/	Divisão
*	Multiplicação
^	Potenciação

Alfabeto da Linguagem

{	Início do escopo.
}	Fim do escopo.
'	Abre / fecha aspas simples.

“	Abre / fecha aspas duplas.
;	Término de um comando.
(	Abre parênteses / entrada de texto.
)	Fecha parênteses / entrada de texto.
	Abre ket.
>>	Fecha ket.
>	Maior que.
<	Menor que.
>=	Maior igual.
<=	Menor igual.
=	Atribuição.
==	Igualdade.
===	Uniformidade.
&&	Operador lógico “e”.
	Operador lógico “ou”.
[	Abre colchetes / vetor.
]	Fecha colchetes / vetor.
:	Atribuição de tipo.
!	Operador lógico “not”.
x	Porta lógica quântica Pauli X. Equivalente operador lógico not.
y	Porta lógica quântica Pauli Y. $ 0\rangle \Rightarrow i 1\rangle$; $ 1\rangle \Rightarrow -i 0\rangle$.
z	Porta lógica quântica Pauli Z. $ 0\rangle \Rightarrow 0\rangle$; $ 1\rangle \Rightarrow - 1\rangle$.
fred	Porta lógica quântica Fredkin. Recebe 3 entradas e 3 saídas. Transmite o primeiro qubit e inverte os outros dois se e somente se o primeiro qubit for 1. Caso contrário, os qubits não são invertidos.
toff	Recebe 3 qubits de entrada e 3 de saída. Se os dois primeiros qubits estiverem em $ 1\rangle$, ele aplica uma operação x no terceiro qubit.

cn	Porta lógica quântica Toffoli. Recebe 3 qubits e aplica a operação x no segundo qubit apenas quando o primeiro $ 1\rangle$.
swap	Porta lógica quântica swap. Transforma $ 00\rangle$ em $ 00\rangle$, $ 01\rangle$ em $ 10\rangle$, $ 10\rangle$ em $ 01\rangle$, $ 11\rangle$ em $ 11\rangle$.
had	Porta lógica hadamard. Atua em um único qubit. Mapeia o estado inicial $ 0\rangle$ para $(0\rangle + 1\rangle) / \sqrt{2}$ e $ 1\rangle$ para $(0\rangle - 1\rangle) / \sqrt{2}$, criando probabilidades iguais do resultado ser 0 ou 1.
sqrnot	Raiz quadrada da porta x.
ps	Porta lógica quântica phase shift. Atua em um único qubit, mantendo $ 0\rangle$ e transformando $ 1\rangle$ em $e^{i\phi} 1\rangle$
sqrSwap	Porta lógica quântica raiz quadrada de swap. Faz metade da troca de dois qubits.
ising	Porta lógica Ising. Usado para fazer a correlação entre os estados do qubits.
qbit	Cria um qubit.

Definições Regulares

[operador] = {+ | - | / | * | ^}

[variável] = {let([A-Za-z0-9])}

[espaçamento] = {[\t \r \n \v \f]}

[condição] = {> | < | == | === | ! | && | || | <= | >= | ! }

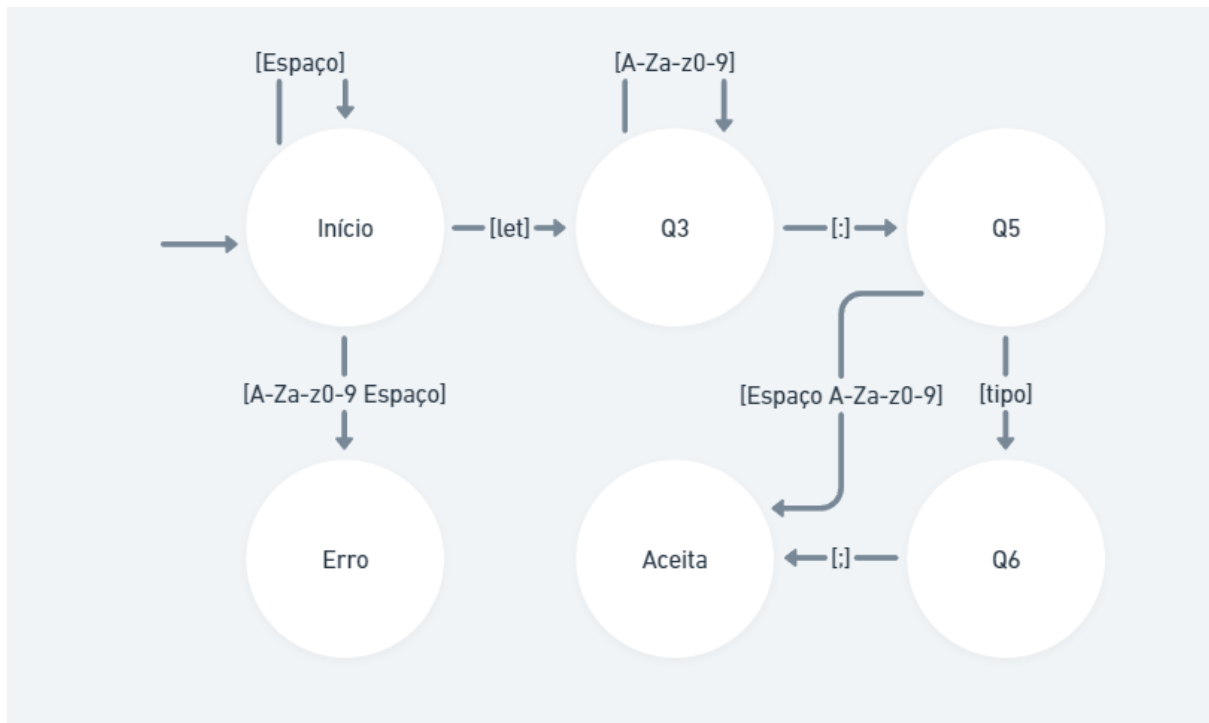
[tipos] = { init | string | bool | float | complex | double }

[símbolos] = { = | { | } | ' | " | ; | < | > | | | >> | [|] | ; | : }

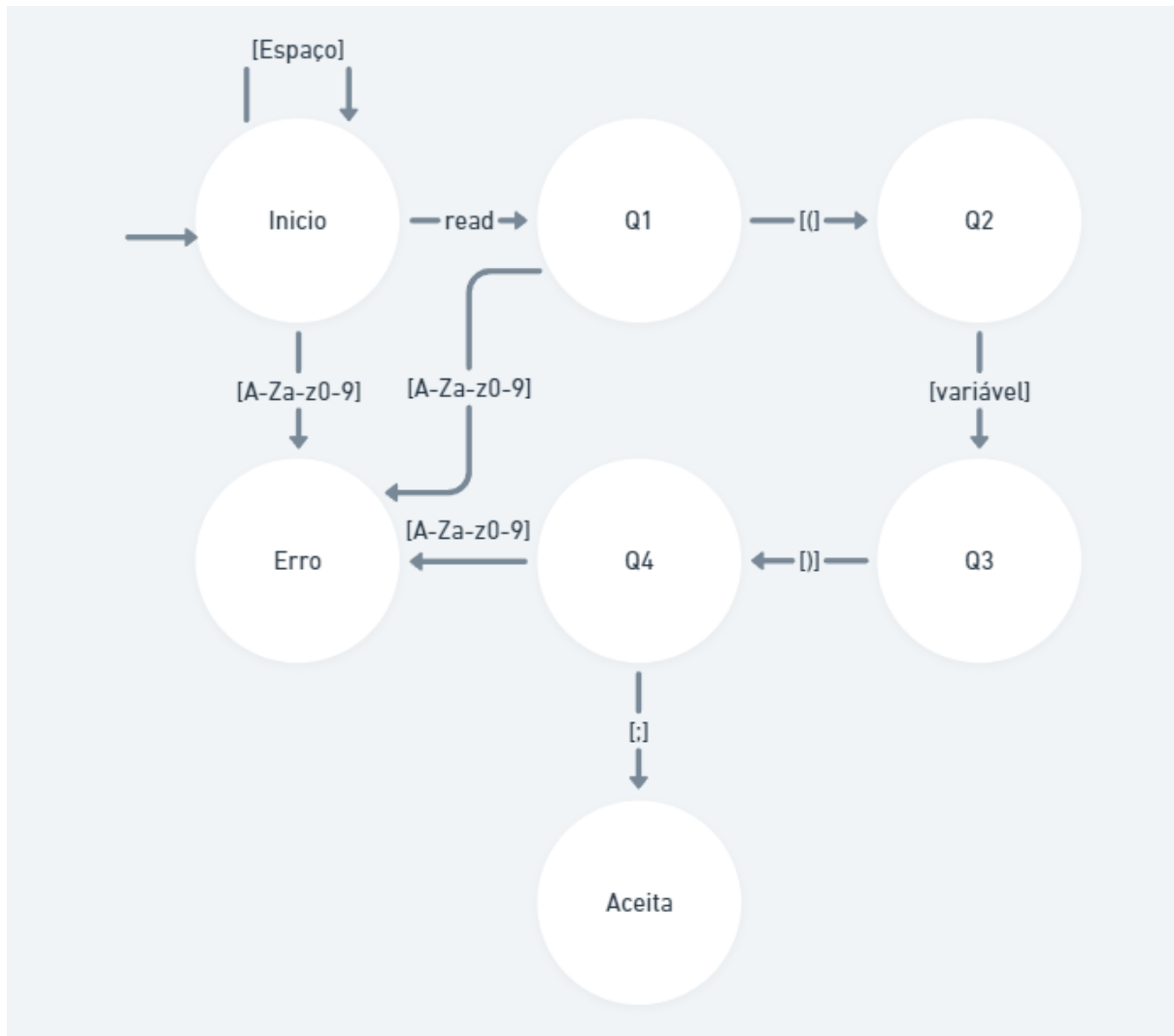
[reservada] = { def | function | print | main | read | x | y | z | fred | toff | cn | had | sqrnot | ps | sqrSwap | ising | qbit | let }

Automatos

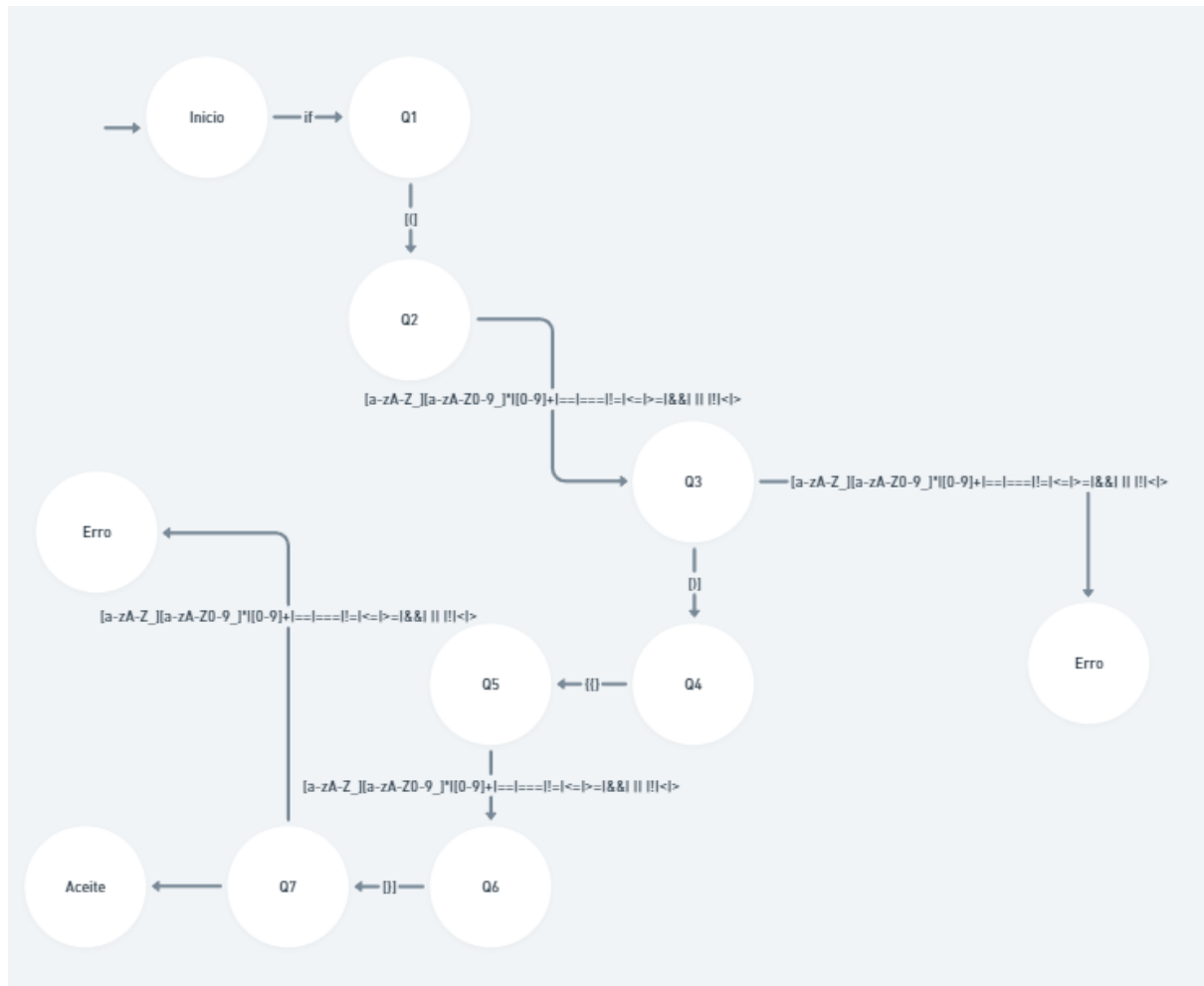
Autômato de definição de variável



Autômato de Leitura



Autômato de Condição if



Referências

O que são linguagens de programação probabilísticas e qual o seu uso. - <https://medium.com/dunnhumby-data-science-engineering/what-are-probabilistic-programming-languages-and-why-they-might-be-useful-for-you-a4fe30c4d409>. Acesso em 15/03/2024.